**S1 Text. Event decoding, account types, balances, the discovery-order protocol, label formulas and extraction queries.**

This text gives what the main text summarizes: which logs were read, how each address was typed, how each field was decoded and valued, how every score input, importance, check and label was computed, how the discovery-order test runs, and which properties of the data were not quantified. Constants are those of the analysis pipeline; the repository and the commits that hold them are named in the Data availability statement. All results of the main text belong to the contract-graph analysis, the main analysis of the article; where the registered analysis differs, the difference is stated.

# 1. Source, extraction and score windows

All records are rows of the public BigQuery table `bigquery-public-data.goog_blockchain_arbitrum_one_us.logs` (Arbitrum One, chain identifier 42161). Each row is one event log with its block timestamp, block number, log index, emitting contract, topics and data. One query (scan A) copied the ERC-20 `Approval` and `Transfer` logs with exactly three topics from 1 October 2023 00:00:00 to 30 June 2026 23:59:59 UTC into a table of the authors' project, partitioned by month. A second query (scan B) did the same for 1 July to 30 September 2026; its script refuses to run unless the W1 registration and the configuration are committed with the status "registered", and it records their SHA-256 hashes in an extraction manifest. Both scans read the whole chain, not the neighbourhood of a cohort.

The contract-graph analysis reads from these two tables the rows whose emitting contract is one of the five tokens of Section 5, from 1 April 2025 00:00:00 to 30 September 2026 23:59:59 UTC: every `Approval`, of any value, and every `Transfer` with a non-zero value, 1,240,275,042 rows with 25,537,280 distinct addresses other than the zero address; 603,475,784 of the rows have a contract at both ends. Two further public sources are read: the public transactions table of the same dataset, for the addresses that sent a transaction (Section 3), and the public logs before 1 October 2023, for the balances (Section 4). Each query was first run as a dry run and checked against a budget; the 26 jobs of this analysis cost USD 12.63, and the jobs of the whole study billed 8.78 TiB and cost USD 54.88.

Every score, check and raw count uses the events of one score window, the twelve months before its freeze (both ends inclusive; each freeze is at 23:59:59 UTC of its date):

| Freeze | Score window | Label window |
|---|---|---|
| $T_{\text{obs}}$, 30 June 2026 (same window and W1) | 1 July 2025 to 30 June 2026 | 1 July to 30 September 2026 |
| $t_1$, 31 March 2026 (W0) | 1 April 2025 to 31 March 2026 | 1 April to 30 June 2026 |

In the registered analysis every score used every extracted event from 1 October 2023 up to its freeze, amounts stayed in raw base units of every ERC-20 token, and the graphs held every account of the events around the plan's cohort.

# 2. Cohorts and spender lists

The plan's candidates are the addresses that received a non-zero approval of any ERC-20 token from at least three distinct owners in scan A; 27,844 of the 29,022 candidates held contract code at block 513,216,506 (9 October 2026). The cohort list of a freeze holds the addresses among these 27,844 that at least three distinct owners approved, with a non-zero `Approval` of one of the five tokens, inside the freeze's score window: 6,483 addresses at $T_{\text{obs}}$ and 6,343 at $t_1$. The rule counts every approving owner, whatever its account type. The cohort of a freeze is its cohort list without the addresses that Section 3 types as user accounts: the matched cohort of $T_{\text{obs}}$ holds 6,443 contracts (40 addresses of the list are smart accounts), and the W0 cohort of $t_1$ holds 6,292 (51 smart accounts); 5,351 contracts are in both.

The spender list of a freeze holds every address that has a positive latest allowance of one of the five tokens at the freeze, set inside the score window, among the approvals that have an address of the freeze's cohort

list as owner or spender. It therefore includes addresses outside the cohort list that a listed address had approved. The spender cohort is the spender list without the addresses that are not contracts: 7,864 of the 7,962 listed spenders at $t_1$ (97 are user accounts and one is the system address `0x...01`) and 7,915 of the 7,995 at $T_{\text{obs}}$ (79 and one). A member of a cohort that ends no pair of a layer of the contract graph stays in its cohort and scores zero under that layer's scores and counts. S1 Table gives the coverage of each cohort.

## 3. Account types

*Rules.* The addresses that are ends of the five-token events of Section 1 are typed in this order; an address that neither sent a transaction nor belongs to the read set of rule 2 is not typed. (1) An address that sent a transaction in any block before 1 October 2026 is a user account (17,477,779 addresses). An account with code cannot send a transaction, apart from an EIP-7702 delegation. (2) Every other address that can be an end of an edge between contracts is read with `eth_getCode` at one pinned block, 513,216,506, the block at which the plan's candidates were read. No code makes it an externally owned account; code consisting of the EIP-7702 delegation designator (`0xef0100` followed by a 20-byte address) makes it an externally owned account as well; both are user accounts. Any other code makes it a candidate contract. (3) A candidate contract is a smart account, and so a user account, if one of two tests holds at the same block. Safe: `getThreshold()` (selector `0xe75235b8`) returns a value from 1 to 1,000 and `getOwners()` (`0xa0e67e2b`) decodes to a non-empty list of addresses. ERC-4337 account: `entryPoint()` (`0xb0d691fe`) returns one of the canonical EntryPoint contracts (versions 0.6 to 0.9), and the code examined contains a `PUSH4` of a `validateUserOp` selector (`0x3a871cdd` or `0x19822f7c`) and none of a `validatePaymasterUserOp` selector (`0xf465c77e`, `0x52b7512c`). For a proxy the code examined is that of its implementation. (4) A candidate contract that is not a smart account is a contract, unless it is one of the system addresses `0x...01` to `0x...ff`, which are never contracts and never nodes.

*Which addresses are read.* The read set holds the addresses that sent no transaction and have at least one counterparty, in either direction and either kind of event, that is neither a transaction sender nor the zero address; every member of the cohort and spender lists; and two audit samples, 1,000 transaction senders and 1,000 addresses outside the set that sent no transaction, each drawn by the lowest SHA-256 hash of a fixed salt and the address. An address outside the set that sent no transaction has only user accounts and the zero address as counterparties, so it can never be the end of an edge between contracts. The set holds 6,230,270 addresses.

*Reads.* Proxies are recognized from the code: an ERC-1167 clone by its code pattern, which carries the implementation (503,516 addresses); an ERC-1967 proxy by the implementation slot constant in its code and a non-zero address in that slot (524,454); a Safe proxy by the `masterCopy()` selector (45,362). An implementation's code is read once. The answers of rule 3 depend on an account's storage for ERC-1967 proxies, Safe proxies and every code whose representative answers `getThreshold()`, so these are probed address by address; every other code hash is probed on its lowest address, and the result holds for every address with that code. A call that the node aborts at its time limit is sent again, up to six answered attempts; a revert or an execution error is the call's own failure. The reads used two public endpoints that serve the pinned block. No address had a call aborted at every attempt, and none of the 1,000 audited transaction senders had code. For the 1,000 audited addresses outside the set, the reads found no code for 758, an EIP-7702 delegation designator for 183, a smart account for 5 and other contract code for 54; such a contract has no contract as counterparty and is no node.

*Code families.* For a proxy whose implementation the reads did not find, rule 3 examines the proxy's own code, which holds no `validateUserOp` selector because the selector sits in the implementation. A report of the largest groups of candidate contracts by code hash shows such ERC-4337 accounts among them. Rule 3 therefore has a second part, which types whole code families and was decided from the code of the families alone, without reading any label, cohort membership or score of the addresses it moves. First, a contract of proxy kind "none" with own code under 1,000 bytes whose code hash answers a canonical EntryPoint and holds no paymaster selector is an ERC-4337 account when its code hash is on a list of 20 code hashes (37,337 contracts). Second, an ERC-1967 proxy whose implementation holds `validateUserOp` and no

paymaster selector and whose `entryPoint()` gives no EntryPoint is an ERC-4337 account when its getter `entrypoint()` (`0xa65d69d4`) answers a canonical EntryPoint; seven pairs of code hash and implementation pass (155,319 contracts). The two lists move 192,656 typed contracts to the user accounts. Counts of the structure pilot's graph with and without the candidate families were known when the lists were drawn; in each of them the protocol's rules set the same reveal mode, schedule and hot-set share, and the stop flag stayed. The lists are published with the definitions of the analysis.

*Result.* Of the 6,230,270 addresses read, 2,909,509 have no code, 1,165,025 an EIP-7702 delegation designator, 46,204 are Safes, 87,157 ERC-4337 accounts by the reads and 192,656 by the listed code families, and 1,829,719 are contracts. With the transaction senders, 21,877,279 addresses are typed user accounts. The contract graph numbers 1,403,959 contracts over both freezes: every contract that ends an allowance pair, with or without the cap, or a transfer pair at either freeze, and the contract members of the cohorts.

*Limits.* The state read is that of one block after the end of the data: an address whose code was deployed only after its events is treated as a contract throughout, and a contract that destroyed itself before EIP-6780 took effect has no code and counts as a user account. Smart accounts that are neither Safes nor ERC-4337 accounts by these tests stay contracts. After the lists, 115,084 typed contracts hold a `validateUserOp` selector and no paymaster selector in the code examined without answering a canonical EntryPoint; 104,187 of them share one code hash and 8,793 one implementation whose getter answers a contract that is not a canonical EntryPoint. Contracts that services create for each user, order or payment are not picked out by any rule; in the report of the largest code families from which the lists were drawn, the seven largest families after the first were of that kind, as inferred from their code. The study does not measure who controls a user account.

## 4. Balances and the balance cap

*Balance.* The balance of a (token, address) pair at a freeze is the net `Transfer` flow of the token to the address up to the freeze, in base units read as double-precision numbers: the public logs before 1 October 2023, the copy of scan A from 1 October 2023 to 31 March 2025 and the five-token rows of Section 1 from 1 April 2025 to the freeze. Balances are computed for every (token, owner) of a five-token approval from 1 April 2025 to $T_{\text{obs}}$, at $t_1$ and at $T_{\text{obs}}$. A negative value, or a value below $10^{-6}$ token, counts as zero.

*Check.* For 1,500 (token, owner) pairs, 150 with a zero and 150 with a positive balance at $T_{\text{obs}}$ for each token, drawn by the lowest SHA-256 hash of a fixed salt and the pair, the reconstructed balance was compared with `balanceOf` at block 479,089,705, the last block of 30 June 2026. A match counts within $10^{-9}$ relative or $10^{-6}$ token. 1,493 pairs match (99.5%; the threshold was 98%); the cells with mismatches are the zero-balance cells of WETH (97.3%) and USDT0 (98.0%). A token whose balances change without `Transfer` logs would not be reconstructed.

*Cap.* For each (token, owner, spender), with $A$ the dollar value of the latest allowance at the freeze day's price and $B$ the dollar value of the owner's balance of the token at the freeze at the same price,

$$a^\star = \min(A, B).$$

The cap applies to the allowances between contracts that form the allowance layer and to the allowances of user accounts that enter the importance $I_A$. An allowance with $a^\star = 0$ is no edge and adds nothing to $I_A$. At $T_{\text{obs}}$, 222,559 of the 339,282 positive latest allowances between contracts are capped to zero and 114,454 are capped below the allowance; 223,892 are unlimited (at least $1.15 \times 10^{77}$ base units). Without the cap the allowance layer would hold 302,559 pairs at $T_{\text{obs}}$ and 275,917 at $t_1$, against 115,006 and 17,940 with it.

## 5. Tokens and prices

*Token rule.* Three conditions admit a token. (i) In the Chainlink directory of Arbitrum One data feeds (cited in the main text), a standard market-price feed in US dollars or in ether exists whose market-risk rating is low or medium. (ii) The feed prices the token itself, an asset that the token's issuer wraps one to one (as WETH wraps ether), or, for a token minted by Arbitrum's canonical bridge, the Ethereum token it represents. (iii) The token appears among the transfers or latest allowances of the events around the plan's cohort up to $T_{\text{obs}}$. Of the 133 tokens that the directory maps, 64 are listed and 46 of these have a daily DefiLlama price. The analysis keeps the five priced listed tokens with the most rows there, which hold 75.75% of the rows of all tokens:

| | | | |
|---|---|---|---|
| WETH | `0x82af49447d8a07e3bd95bd0d56f35241523fbab1` | 18 decimals | feed ETH / USD |
| USDC | `0xaf88d065e77c8cc2239327c5edb3a432268e5831` | 6 decimals | feed USDC / USD |
| USDT0 | `0xfd086bc7cd5c481dcc9c85ebe478a1c0b69fcbb9` | 6 decimals | feed USDT / USD |
| ARB | `0x912ce59144191c1204e64559fe8253a0e49e6548` | 18 decimals | feed ARB / USD |
| WBTC | `0x2f2a2543b76a4166549f7aab2e75bef0aefc5b0f` | 8 decimals | feed WBTC / USD |

All five feeds are rated low market risk. Events of every other token, including bridged USDC.e, are not read.

*Prices.* The price of a token on a UTC day is the point of DefiLlama's daily chart under the token's Arbitrum address that lies nearest to 00:00 UTC of the following day. Each of the 548 days from 1 April 2025 to 30 September 2026, which the score and label windows cover, is priced under the token's own address. A point more than twice or less than half of both neighbouring points, while those lie within a factor of two of each other, would be taken from the previous day, and a day without a point would take the previous day's price; neither case occurs for the five tokens on these days. On 20 of these days per token the DefiLlama price was compared with the Chainlink answer in force at 00:00 UTC of the next day, read on the chain: the median absolute relative difference is 0.010% to 0.056% per token and the largest 0.24%, for WBTC.

*Valuation.* The dollar value of a transfer is $x = v\,10^{-q}p$, with $v$ its amount in base units, $q$ the token's decimals and $p$ the token's price on the transfer's UTC day. Latest allowances and balances are valued at the price on the day of the freeze (30 June 2026 for $T_{\text{obs}}$, 31 March 2026 for $t_1$). The 32-byte amount is read as a double-precision number.

## 6. ERC-20 approvals and the allowance layer

*Selection and fields.* Topic 0 equals `0x8c5be1e5ebec7d5bd14f71427d1e84f3dd0314c0f7b2291e5b200ac8c7c3b925`, the keccak-256 hash of `Approval(address,address,uint256)`, and the log has exactly three topics, which excludes ERC-721 logs (they index the token identifier as a fourth topic). Topic 1 is the owner and topic 2 the spender (the last 20 bytes of each 32-byte word); the emitting contract is the token; the data field is the allowance. An EIP-2612 `permit` emits the same event and is therefore included. An approval of zero is kept, because it sets the allowance in force to zero. A data field that cannot be read as a number gives zero.

*Latest allowance and edge.* For each (token, owner contract, spender contract) triple the record with the largest (block number, log index) inside the score window, up to the freeze, is kept when its value is positive. With $\Delta t^\star$ the age in days of the kept record at the freeze and $a^\star$ its capped dollar value (Section 4), a triple with $a^\star > 0$ is an allowance of the layer, and the EndorseRank weight of the pair of owner $u$ and spender $v$ is

$$w_A(u,v) = \sum_{\text{token}} \sigma(\Delta t^\star)\, V_A(a^\star), \qquad \sigma(\Delta t) = \frac{1}{1+e^{0.01(\Delta t - 180)}},$$

$$V_A(z) = \frac{2}{1+e^{-b_A z}} - 1, \qquad b_A = \frac{\ln 3}{0.071} \approx 15.47.$$

In floating point $V_A$ is set to one from $b_A z \geq 40$, and a pair whose weight is zero in double precision is not walked: at $T_{\text{obs}}$, 114,997 of the 115,006 pairs have a positive weight. The amount-weighted allowance layer of the coupled walk sums $\min(a^\star, U)$ over tokens with $U = 16{,}000$ dollars. The in-approve degree of a contract is the number of owner contracts with a positive capped latest allowance to it, and its in-approve value is the sum of $\min(a^\star, U)$ over its owner contracts and tokens.

*Constants.* The constants are computed once, on the W0 score window of the contract graph, rounded to two significant digits and used at both freezes. $m_A = 0.071$ dollars is the median of the positive capped values $a^\star$ of the latest allowances between contracts at $t_1$, unlimited allowances included (0.0707 dollars before rounding; the 10th and 90th percentiles are $3.0 \times 10^{-6}$ and 26.4 dollars). $m_T = 160$ dollars is the median dollar value of the transfers between contracts in that window (164.05 dollars), and $U$ is the 99th percentile of the same values (16,412.3 dollars). $U_{\text{user}} = 31{,}000$ dollars is the 99th percentile of the dollar values of the transfers from user accounts to contracts in that window (31,306.6 dollars). None uses an event after $t_1$, so no label window enters them.

*Known properties not quantified in this study.* (a) Many ERC-20 implementations also emit `Approval` from inside `transferFrom` when they reduce a finite allowance. For such tokens the kept record may be the remainder after the spender's last use, not a new authorization: its age then dates the last spend, an allowance spent in full ends at zero and is no edge, and an authorization from before the score window that is used during a label window can be counted as a new approval pair. The share of `Approval` logs emitted in the same transaction as a `Transfer` by the same token with the same owner and spender would measure this; it was not computed. (b) Contracts that manage allowances of their own, such as Uniswap's Permit2, record those inner allowances in events with a different signature, which the extraction does not read. Owners approve such a manager through an ordinary ERC-20 approval, so it appears as one spender, and the contracts it authorizes in turn are not seen. (c) Only the approvals inside a freeze's score window are read for its graph: an allowance that is still in force at the freeze and was last approved before the window opened has no edge. (d) The balance cap uses the owner's balance at the freeze; an owner contract that holds a token only for the length of a transaction has no balance at the freeze, and its allowance is capped to zero.

# 7. ERC-20 transfers and the transfer layer

Topic 0 equals `0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef`, the keccak-256 hash of `Transfer(address,address,uint256)`, with exactly three topics. Topic 1 is the sender, topic 2 the recipient and the data field the amount. Zero-value transfers are not read. A transfer is an edge of the layer when its sender and its recipient are both contracts; mint and burn logs have the zero address at one end and are no edge. A self-transfer becomes a self-loop in the walks and is excluded from the transfer in-degree, from the union graph and from the new-sender label. The AWP weight and restart weight are

$$w_T(u,v) = \sum_{e:\, u \to v} \sigma(\Delta t_e)\, V_T(x_e), \qquad X_u = \sum_{v \neq u} \max_{e:\, u \to v} \sigma(\Delta t_e),$$

$$V_T(z) = \frac{2}{1 + e^{-b_T z}} - 1, \qquad b_T = \frac{\ln 3}{160} \approx 0.0069.$$

Only transfers inside the score window enter the weights, and $\Delta t_e$ runs from 0 to 365 days. The amount-weighted transfer layer of the coupled walk sums $x_e\, \sigma(\Delta t_e)$. A spender that moves tokens with `transferFrom` directly from an owner to a third address does not appear in the resulting `Transfer` event.

# 8. The discovery-order test in detail

The protocol of the test (`docs/discovery_protocol.md` of the experiment folder, published with the results) states the following; the main text gives the question, the statistic and the decision. The test was defined after the registered results were known and decides no rule of the analysis plan or the registration.

*Graph and universes.* $G$ is the contract graph of a freeze with both layers and their weights. The union graph holds the distinct pairs $(p, q)$, $p \neq q$, with an edge in either layer; the bow-tie, the universes and the hot sets are defined on it, and the walks keep self-loops as the solver does. In the `out` mode the universe is the giant strongly connected component (of two equally large, the one with the lowest address) with every contract reachable from it. In the `both` mode an edge is revealed at the first visit of either of its ends, the universe is the weakly connected component that holds the giant component, and the row of a frontier contract in a walk is normalized by its revealed, partial out-strength. A structure pilot at $t_1$ read the pairs of each layer without weights and reported the sizes of the layers, the bow-tie, the two universes, the share of the `out` universe with a capped allowance in-edge, and the largest groups of contracts by code hash; it computed no weight, importance, crawl or ranking. Three rules were applied to it and used at both freezes: the `both` mode is primary if the `out` universe holds fewer than half the contracts of the `both` universe (332,396 against 1,015,867); $\rho$ is the smallest of 0.01, 0.02 and 0.05 whose projected run time is at most 12 hours (projected 12.49, 6.75 and 2.98 hours, so $\rho = 0.02$); and the hot share $q$ is 1% if the primary universe holds at least 10,000 contracts. The protocol stops the run if the capped allowance pairs between contracts at $t_1$ number fewer than 5,000 or if fewer than 5% of the contracts of the `out` universe have a capped allowance in-edge. The pilot counted 17,905 such pairs and a share of 0.54%, so the second flag was raised. The test went on by a decision of the first author, recorded with the protocol, with these reasons: no setting that the protocol allows lifts the flag; the test is the only direct test of its question; and the part of the allowance layer in the result is read from the contrasts that the protocol reports outside the decision.

*Seeds.* The 20 contracts of the giant strongly connected component with the lowest SHA-256 hash of the string `crawl-seed-v1:` followed by the address. The seed is visit 1, and every crawl ends with its universe visited.

*Schedule.* After $V$ visits the ordering is recomputed; then the $\min(b(V), F)$ highest-scored contracts of the frontier are visited in order, with $b(V) = \max(1, \lceil \rho V \rceil)$ and $F$ the number of frontier contracts. The breadth-first order does not depend on $\rho$, and the random order draws one frontier contract uniformly at every visit from a generator seeded by the seed address.

*Orderings.* BFS visits the frontier in increasing index of discovery. The counts on the visited part are $\mathrm{IB}'_A(f)$, the distinct visited contracts with an allowance edge to $f$; $\mathrm{IB}'_T(f)$, the distinct visited contracts other than $f$ with a transfer edge to $f$; and $\mathrm{IB}'_U(f)$, the distinct visited contracts other than $f$ with an edge to $f$ in either layer. The walks run on the seen graph, which holds the visited and frontier contracts and the revealed edges with their weights: EndorseRank$'$ (uniform restarts over the seen graph), EndorseRank-activity$'$ and AWP$'$ (restarts by the activeness computed on the revealed edges), C-PR$'$ at $\lambda = 0.5$, 0.25 and 0.75, its layers C-PR$'(\lambda{=}1)$ and C-PR$'(\lambda{=}0)$, and S-PR$'$. In the `out` mode a frontier contract has no revealed out-edge, so its mass returns through the restarts. The solver is that of the main text ($d = 0.85$, tolerance $10^{-8}$, at most 300 iterations), and each recompute starts from the previous vector on the old nodes. The frontier, sorted by decreasing score, is cut into groups of scores within $10^{-12}$ times the frontier maximum of the group's highest score; inside a group the contract discovered first goes first, then the lower address, and a pick counts as decided by a tie when its group holds more than one contract. The references that are not crawls are the ideal order (hot contracts first, by decreasing importance) and the orders by decreasing C-PR ($\lambda = 0.5$) and by decreasing union in-degree of the full graph of the universe. The diagnostic $\delta'(f)$ is the sum over visited contracts $v$ with an edge to $f$ of the weight of that edge divided by the out-strength of $v$, on the mixed weights at $\lambda = 0.5$.

*Importance.* User accounts are the accounts of Section 3; no ordering sees any of their edges. For each contract $c$ of the universe, over the score window, $I_T(c)$ sums over user accounts $w$ the dollars that $w$ transferred to $c$, each account counted up to $U_{\mathrm{user}}$, and $I_A(c)$ sums over user accounts the capped dollar value $a^\star$ of the latest allowances of $w$ to $c$ over the five tokens, each account counted up to $U_{\mathrm{user}}$. $N_T(c)$ and $N_A(c)$ count the user accounts with at least one dollar transferred or authorized. At $T_{\mathrm{obs}}$, 809,296 typed contracts have a positive importance and 52,446 a positive $I_A$; the sums are 16,611,212,142 dollars of $I_T$ and 309,472,584 dollars of $I_A$, of which the universe of the deciding run holds 96.5% and 93.0%. User accounts also transferred 1,681,953,612 dollars to addresses that were not typed because every counterparty of theirs is a user account; these addresses are never nodes.

*Statistic, curves and uncertainty.* With $t_h$ the visit of hot contract $h$, $g_x(h) = \max(0, 1 - t_h / \lceil x|\mathcal{U}| \rceil)$ is

averaged over the 20 seeds and then over the hot set to give $\mathrm{AUC}_x$. The discovery curve is $P_{ST}(K) = |H \cap V_K|/|H|$, with $V_K$ the first $K$ contracts visited, on a grid of 1,000 values of $K$, as the mean, minimum and maximum over the seeds; the ideal curve is $\min(1, K/|H|)$ and the random curve $K/|\mathcal{U}|$. The cumulative captured importance is the share of the summed importance of the universe held by the contracts visited so far. Intervals come from a paired bootstrap over the hot contracts with 400 resamples and seed 42, one index matrix shared by every ordering and contrast; the seeds are fixed in it, and the spread across seeds and a bootstrap over the seeds are reported as sensitivities.

*Decision and what is reported outside it.* As in the main text. At $t_1$ and in the other mode the same contrasts are reported without a verdict. The no-hubs sensitivity takes the $\max(10, \lceil 0.001|\mathcal{U}| \rceil)$ contracts with the highest union in-degree of the full graph (1,170 in the deciding run) out of the hot sets and keeps them in the graph. The schedule sensitivity runs $\rho/4$ on the first five seeds.

*Run.* The four runs (two freezes, two modes) were made by one script, which records the SHA-256 hashes of the protocol, the definitions of the analysis, the configuration and the code in its output. The deciding run took 29,948 seconds with 16 workers. The published summaries hold the sizes, the hot sets, the area of every ordering with its per-seed values and intervals, the curves, the contrasts, the diagnostics and the run times; the visit of every hot contract for every seed and ordering is published as parquet files.

# 9. Label formulas

*Spenders.* New approval pairs of spender $v$: the number of distinct owners $u$ that emit a positive `Approval` of one of the five tokens for $v$ inside the label window while the pair $(u, v)$ held no positive latest allowance, set inside the score window, at the freeze. New transfer senders of $v$: the number of distinct addresses $u \neq v$ that send $v$ a transfer of one of the five tokens inside the label window and sent it none between the start of the score window and the freeze. Both are counted on events with an address of the freeze's cohort list at one end, and both count owners and senders of every account type. Each label was also split by the type of the account it counts: in W0, user accounts are 676,525 of the 748,258 new approval pairs and 562,007 of the 651,474 new senders; in W1, 617,088 of 702,275 and 509,194 of 618,461.

*Further spender counts.* Two counts that the plan did not name were computed on the same spenders; the main text reports them without offering them as evidence. Revocations of $v$: the number of owners $u$ whose pair $(u, v)$ held a positive latest allowance at the freeze and who emit an `Approval` of zero for $v$ inside the label window. Drained owners of $v$: the number of owners $u$ that give $v$ a positive approval inside the window after which, within six hours, an outgoing transfer from $u$ in the same token moves at least half the approved amount; for an approval of $10^{30}$ base units or more, any outgoing transfer of that token from $u$ within one hour counts. Both use the five tokens only.

# 10. Same-window checks

For a contract $w$ of the matched cohort, with $\mathcal{T}_{\mathrm{in}}(w)$ the transfers that it received from contracts in the score window of $T_{\mathrm{obs}}$ (self-transfers included) and $u_e$ the sender of transfer $e$: in-degree $= |\{u_e\}|$; in-value $= \sum_e x_e$ in dollars; in-approve degree and in-approve value as in Section 6, counted on the capped allowance layer. The Sybil-stability family uses the ratio of distinct inbound counterparties to inbound transfers from other contracts; the transfer tenure, the span in days between the first and the last transfer attributed to $w$ (a transfer between two cohort contracts is attributed to its sender); and the number of active months, the larger of the months in which $w$ received a transfer from another contract and the months of the transfers attributed to it. Inside the score window the tenure is at most 365 days and the active months at most 12. A contract absent from a check's source is given zero. Each family mean is the plain average of the Kendall coefficients of its checks.

# 11. Query patterns

The listings are abridged; the full files are in `sql/` of the experiment folder. Names that start with `@` are query parameters, and `u256` reads a 32-byte amount as a double-precision number.

```
-- sql/24_c12_logs5.sql: the five-token events of both score windows and
-- both label windows, from the copies of scan A (logs_obs) and scan B
-- (logs_w1); every Approval, and every Transfer with a non-zero value
SELECT block_timestamp, block_number, log_index, kind, emitter AS token,
       a, b, u256(val) AS value            -- a: owner or sender, b: spender
FROM `contract_rep.logs_obs`               --    or recipient
WHERE block_timestamp >= TIMESTAMP(@start_ts)
  AND block_timestamp <  TIMESTAMP(@mid_ts) AND kind IN ('A', 'T')
  AND emitter IN (SELECT token FROM `contract_rep.u_tokens`)
  AND (kind = 'A' OR LTRIM(TO_HEX(val), '0') != '')
UNION ALL ...                              -- the same on logs_w1
-- usd of a transfer: value / POW(10, decimals) * price of its UTC day

-- sql/26_c12_tx_senders.sql: the ends that sent a transaction
SELECT DISTINCT FROM_HEX(SUBSTR(t.from_address, 3)) AS address
FROM `bigquery-public-data.goog_blockchain_arbitrum_one_us.transactions` AS t
WHERE t.block_timestamp < TIMESTAMP(@end_ts)
  AND FROM_HEX(SUBSTR(t.from_address, 3))
      IN (SELECT address FROM `contract_rep.c12_ends`);

-- sql/31_c12_contracts.sql: the node set and the events between contracts
CREATE TABLE `contract_rep.c12_contracts` AS
SELECT t.address FROM typed AS t           -- the code reads, with the two
LEFT JOIN `contract_rep.c12_tx_senders` AS s ON s.address = t.address
WHERE t.account_class = 'contract'         -- lists of code families applied
  AND s.address IS NULL AND t.address != FROM_HEX(REPEAT('00', 20))
  AND t.address NOT BETWEEN @system_first AND @system_last;
CREATE TABLE `contract_rep.c12_events_cc` AS
SELECT l.* FROM `contract_rep.c12_logs5` AS l
WHERE l.a IN (SELECT address FROM `contract_rep.c12_contracts`)
  AND l.b IN (SELECT address FROM `contract_rep.c12_contracts`);

-- sql/33_c12_bal.sql: balance of (token, owner) at a freeze = net Transfer
-- flow up to the freeze: public logs before scan A (sql/32), scan A up to
-- the start of the events above, then those events
SELECT token, address, SUM(d) AS net       -- d = +value for the recipient,
FROM flows GROUP BY token, address;        --     -value for the sender

-- sql/36_c12_allow_triples.sql: latest allowance between contracts in the
-- score window, valued at the freeze day's price and capped at the balance
SELECT token, owner, spender,
       value / POW(10, decimals) * price_usd AS usd_allow,
       LEAST(value / POW(10, decimals) * price_usd,
             IF(bal >= POW(10, decimals - 6), bal, 0)
               / POW(10, decimals) * price_usd) AS usd_eff
FROM ( SELECT *, ROW_NUMBER() OVER (PARTITION BY token, a, b
            ORDER BY block_number DESC, log_index DESC) AS rn
       FROM `contract_rep.c12_events_cc`
       WHERE kind = 'A' AND block_timestamp >= TIMESTAMP(@start_ts)
         AND block_timestamp <= TIMESTAMP(@anchor_ts) )
WHERE rn = 1 AND value > 0;

-- sql/37_c12_pairs.sql: pair weights of the two layers at a freeze
```

```
SELECT owner, spender,
       SUM(sig(age_days) * vt(usd_eff, @b_allowance)) AS w_er,  -- EndorseRank
       SUM(LEAST(usd_eff, @cap_allowance))            AS w_raw  -- C-PR layer
FROM `contract_rep.c12_allow_triples` WHERE usd_eff > 0
GROUP BY owner, spender;
SELECT from_address, to_address,
       SUM(sig(age_days) * vt(usd, @b_transfer)) AS w_awp,      -- AWP
       SUM(usd * sig(age_days)) AS w_raw,                       -- C-PR layer
       MAX(sig(age_days)) AS max_sig                            -- activeness
FROM transfers_between_contracts_in_the_score_window
GROUP BY from_address, to_address;

-- sql/41_c12_importance.sql: importance of contract c from user accounts w
--   I_T(c) = SUM over w of LEAST(dollars w transferred to c, @cap_user)
--   I_A(c) = SUM over w of LEAST(SUM over tokens of usd_eff(w, c), @cap_user)
-- with usd_eff the capped latest allowance of w to c, as in sql/36
```

The scan dates and the extraction times of the July–September logs are recorded in the pipeline's extraction manifests. The extraction script of scan B started at 16:49:33 UTC on 9 October 2026, one second after the registration commit, and created the scan B query at 16:49:40, eight seconds after it. Before the tables of the contract graph were downloaded, 64 invariants were checked on them, each a count of violations that must be zero, among them that every node and every end of a pair is a contract and that no user account is the end of a pair; all were zero.